LANGUAGES AND AUTOMATA I

HOMEWORK #4 PARTIAL ANSWERS

string_stack.c

```c
#include "string_stack.h"

#include <string.h>
#include <stdlib.h>
#include <math.h>

const int INITIAL_CAPACITY = 16;

struct _Stack {
    char** elements; // array of strings
    int capacity;    // can grow and shrink
    int top;         // index of next slot to fill, also the size
};

stack_response create() {
    stack s = malloc(sizeof(struct _Stack));
    if (s == NULL) {
        return (stack_response){out_of_memory, NULL};
    }
    s->capacity = INITIAL_CAPACITY;
    s->top = 0;
    s->elements = malloc(INITIAL_CAPACITY * sizeof(char*));
    if (s->elements == NULL) {
        return (stack_response){out_of_memory, NULL};
    }
    return (stack_response){success, s};
}

int size(const stack s) {
    return s->top;
}

bool is_empty(const stack s) {
```

```
    return s->top == 0;
}

bool is_full(const stack s) {
    return s->top == MAX_CAPACITY;
}

response_code push(stack s, char* item) {
    if (is_full(s)) {
        return stack_full;
    }
    if (s->top == s->capacity) {
      // Double the array size, but don't exceed the max capacity
      int new_capacity = s->capacity * 2;
      if (new_capacity > MAX_CAPACITY) {
          new_capacity = MAX_CAPACITY;
      }
      char** new_elements = realloc(s->elements, new_capacity * sizeof(char*));
      if (new_elements == NULL) {
          return out_of_memory;
      }
      s->elements = new_elements;
      s->capacity = new_capacity;
    }
    // Not too many folks know about strnlen, but it's great.
    size_t item_length = strnlen(item, MAX_ELEMENT_BYTE_SIZE + 1);
    if (item_length > MAX_ELEMENT_BYTE_SIZE) {
        return stack_element_too_large;
    }
    // Defensive copy! The stack is going to own a new copy of the
    // string, so that the caller can not manipulate the stack contents
    // through a back door. It is safe to use strdup here since we
    // previously checked its length.
    s->elements[s->top++] = strdup(item);
    return success;
}

string_response pop(stack s) {
    if (is_empty(s)) {
        return (string_response){stack_empty, NULL};
    }
    char* popped_value = s->elements[--s->top];
    // Unhook the string from the stack because the "owner" of the string's
    // memory is going to be the response object. Do we really have to do this?
    // It's good practice!
    s->elements[s->top] = NULL;
```

```
    if (s->top < s->capacity / 4) {
      // Shrink the array, but don't go below the initial capacity
      int new_capacity = s->capacity / 2;
      if (new_capacity < INITIAL_CAPACITY) {
          new_capacity = INITIAL_CAPACITY;
      }
      char** new_elements = realloc(s->elements, new_capacity * sizeof(char*));
      if (new_elements == NULL) {
          return (string_response){out_of_memory, NULL};
      }
      s->elements = new_elements;
      s->capacity = new_capacity;
    }
    return (string_response){success, popped_value};
}

void destroy(stack* s) {
    // Both the array and the stack itself were malloc'd
    free((*s)->elements);
    free(*s);
    *s = NULL;
}
```

stack.h

```
// A class for an expandable stack. There is already a stack class in the
// Standard C++ Library; this class serves as an exercise for students to
// learn the mechanics of building generic, expandable, data structures
// from scratch with smart pointers.

#include <stdexcept>
#include <string>
#include <memory>
using namespace std;

// A stack object wraps a low-level array indexed from 0 to capacity-1 where
// the bottommost element (if it exists) will be in slot 0. The member top is
// the index of the slot above the top element, i.e. the next available slot
// that an element can go into. Therefore if top==0 the stack is empty and
// if top==capacity it needs to be expanded before pushing another element.
// However for security there is still a super maximum capacity that cannot
// be exceeded.

#define MAX_CAPACITY 32768
#define INITIAL_CAPACITY 16
```

```
template <typename T>
class Stack {
  // Allocate the data array with a smart pointer, so no destructor needed
  unique_ptr<T[]> elements;
  int capacity;
  int top;

  // Prohibit copying and assignment
  Stack(Stack&) = delete;
  Stack& operator=(Stack&) = delete;

public:
  Stack():
    top(0),
    elements(make_unique<T[]>(INITIAL_CAPACITY)),
    capacity(INITIAL_CAPACITY) {
  }

  int size() {return top;}

  bool is_empty() {return top == 0;}

  bool is_full() {return top == MAX_CAPACITY;}

  void push(T item) {
    if (top == MAX_CAPACITY) {
      throw overflow_error("Stack has reached maximum capacity");
    }
    if (top == capacity) {
     // No more room, expand if we can
     reallocate(capacity * 2);
    }
    // We have to assume that the type T has a proper copy constructor
    // and copy assignment operator.
    elements[top++] = item;
  }

  T pop() {
    if (is_empty()) {
      throw underflow_error("cannot pop from empty stack");
    }
    if (top < capacity / 4) {
      // Too much empty space, contract if we can
      reallocate(capacity / 2);
    }
    T poppedValue = elements[--top];
```

```
    // For security, blank out the cell holding the popped value so that the
    // old data does not hang around. "Blanking out" in this case just means
    // assigning the default value of the type T to the cell.
    elements[top] = T();
    // Again, assume that the type T has a proper copy constructor and copy
    // assignment operator.
    return poppedValue;
  }

private:
  void reallocate(int new_capacity) {
      // Prevent capacity from going below initial or above maximum.
      capacity = max(new_capacity, INITIAL_CAPACITY);
      capacity = min(new_capacity, MAX_CAPACITY);

      // Allocate a new array with the new capacity.
      unique_ptr<T[]> new_elements = make_unique<T[]>(capacity);

      // Copy the elements in the old array to the new array.
      std::copy(elements.get(), elements.get() + top, new_elements.get());

      // The stack needs to point to the new array. The old array will be
      // automatically deleted when reassigned. But we can't just do
      // elements = new_elements; because elements is a unique_ptr, so we have to
      // use the move() function to transfer ownership of the new array
      // to the stack.
      elements = std::move(new_elements);
  }
};
```

lib.rs

```
pub struct Stack<T> {
    // stack items are private by default
    items: Vec<T>,
}

impl<T> Stack<T> {
    pub fn new() -> Self {
        Stack { items: Vec::new() }
    }

    pub fn push(&mut self, item: T) {
        self.items.push(item);
    }
```

```
    pub fn pop(&mut self) -> Option<T> {
        self.items.pop()
    }

    pub fn peek(&self) -> Option<&T> {
        self.items.last()
    }

    pub fn is_empty(&self) -> bool {
        self.items.is_empty()
    }

    pub fn len(&self) -> usize {
        self.items.len()
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_push_and_pop() {
        let mut stack: Stack<i32> = Stack::new();
        assert!(stack.is_empty());
        stack.push(1);
        stack.push(2);
        assert_eq!(stack.len(), 2);
        assert_eq!(stack.pop(), Some(2));
        assert_eq!(stack.pop(), Some(1));
        assert_eq!(stack.pop(), None);
        assert!(stack.is_empty());
    }

    #[test]
    fn test_peek() {
        let mut stack: Stack<i32> = Stack::new();
        assert_eq!(stack.peek(), None);
        stack.push(3);
        assert_eq!(stack.peek(), Some(&3));
        stack.push(5);
        assert_eq!(stack.peek(), Some(&5));
    }

    #[test]
    fn test_is_empty() {
```

```
        let mut stack: Stack<String> = Stack::new();
        assert!(stack.is_empty());
        stack.push(String::from("hello"));
        assert!(!stack.is_empty());
        stack.pop();
        assert!(stack.is_empty());
    }

    #[test]
    fn test_stacks_cannot_be_cloned_or_copied() {
        let stack1: Stack<i32> = Stack::new();
        let _stack2: Stack<i32> = stack1;
        // Should get a compile error if next line uncommented
        // let _stack3: Stack<i32> = stack1; // Error: `stack1` has been moved
    }
}
```